

Non-Parametric Methods

Polynomial Regression, Basis Expansion, and the Curse of Dimensionality

Tu Li

Fall 2026

Econ 453 University of Arizona

Roadmap

1. Parametric vs Non-Parametric
2. First Attempt — Minimising Over a Function Space
3. Polynomial Spaces
4. Why Polynomials? — Two Justifications
5. Polynomial Regression in Practice
6. Multivariate and the Curse of Dimensionality
7. General Basis Function Approach
8. What We Skipped
9. OLS vs Shrinkage vs Polynomial Regression
10. Summary

1. Parametric vs Non-Parametric

1.1 Everything So Far Has Been Parametric

Take stock of the methods we have built:

- **Normal model** (L2): once we know μ and σ^2 , the distribution is pinned down.
- **OLS / linear regression** (L6–L9): once we know $\beta \in \mathbb{R}^{p+1}$, the regression function $f(\mathbf{x}) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$ is pinned down.
- **Ridge and LASSO** (L10): same coefficient vector, with a penalty or constraint applied.

In every case, “find the function” was reduced to “find a finite vector of numbers” — a **parameter**. Different settings, same shape of problem.

1.2 Definition — Parametric Model

Definition — parametric model

A **parametric model** is a family of functions or distributions indexed by a *finite-dimensional* parameter $\theta \in \mathbb{R}^k$:

$$\mathcal{F} = \{ f_{\theta} : \theta \in \mathbb{R}^k \}.$$

Once θ is fixed, the model is fully specified.

Notation reminder. θ is the parameter; the collection $\mathcal{F}$ is the **model class**; k is the *dimension* of the model (number of free numbers).

Why this matters. Estimating a parametric model means estimating k numbers — a problem we know how to handle (OLS, MLE, ridge, LASSO).

1.3 Three Examples of Parametric Models

Example 1 — Normal $N(\mu, \sigma^2)$

Family: all Normal distributions on $\mathbb{R}$. Parameter:

$\theta = (\mu, \sigma^2) \in \mathbb{R} \times \mathbb{R}_{>0}$. Dimension $k = 2$.

Example 2 — Linear regression

Family: all linear functions of p predictors. Parameter: $\theta = \beta \in \mathbb{R}^{p+1}$.

Dimension $k = p + 1$.

Example 3 — LASSO

Same coefficient space $\beta \in \mathbb{R}^{p+1}$, but constrained by $\sum_{j=1}^p |\beta_j| \leq c$ for some hyperparameter c . The intercept β_0 is not penalised. Still parametric: the unknown is a finite vector. The ℓ_1 constraint shrinks the feasible set, it does not change the dimension of θ .

In all three, k *does not depend on the sample size* n . Add another row of data — θ stays the same shape.

1.4 Definition — Non-Parametric Model

Definition — non-parametric model

A **non-parametric model** is one whose unknown object cannot be described by a finite-dimensional parameter. The unknown is the function f itself:

$$\mathcal{F} = \{ f : f \in \text{some infinite-dimensional space} \}.$$

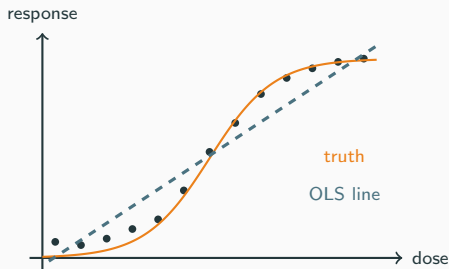
“Non-parametric” is mildly misleading — there are still parameters, just *infinitely many of them*. A better mental picture: we let the data, not a fixed formula, decide the shape of f .

The challenge. If $\mathcal{F}$ is too large, the “estimate” is ill-defined (§2). The art of non-parametric regression is choosing a $\mathcal{F}$ that is *rich enough* to capture the truth and *small enough* to be estimable from n observations.

1.5 When We Need It

Linear regression fits a *straight line* (or a flat hyperplane). When the truth is bent, OLS misses badly:

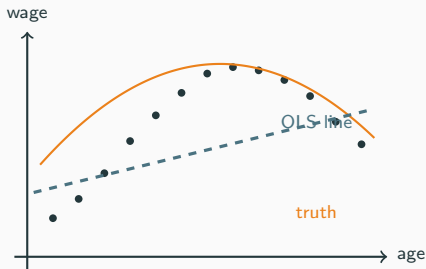
Example 1 — Dose–response (S-curve).



1.5 When We Need It 2

Drug response saturates — a straight line under- and over-shoots.

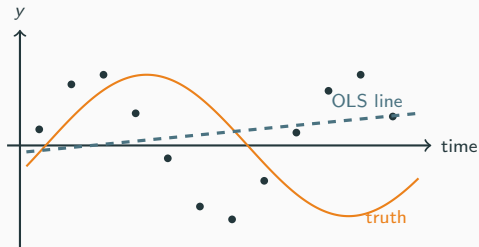
Example 2 — Wage as a function of age.



Wages rise, plateau, then decline — a straight line cannot bend.

Example 3 — Cyclical / seasonal data.

1.5 When We Need It 3



Seasonal pattern, linear fit captures none of it.

In each case the OLS line (dashed) is roughly flat — on average right, but wrong everywhere. We need a method that lets the fitted curve *bend*.

2. First Attempt — Minimising Over a Function Space

2.1 The Natural Generalisation

Recall the OLS recipe from L9. With p predictors, define the intercept-augmented vector $\tilde{\mathbf{x}}_i = (1, x_{i1}, \dots, x_{ip})^\top \in \mathbb{R}^{p+1}$:

$$\hat{\beta} = \arg \min_{\beta \in \mathbb{R}^{p+1}} \sum_{i=1}^n (y_i - \tilde{\mathbf{x}}_i^\top \beta)^2.$$

The decision variable is a vector in $\mathbb{R}^{p+1}$.

The natural extension. If the target is the regression function f itself, just *swap the search set* from $\mathbb{R}^{p+1}$ to a *space of functions* $\mathcal{F}$:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \sum_{i=1}^n (y_i - f(x_i))^2.$$

Same loss, same minimisation, just a richer decision variable.

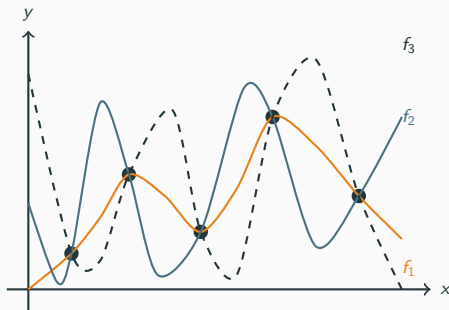
Does this work? Not as stated.

2.2 The Problem — Too Many Solutions

Let $\mathcal{F}$ be “all continuous functions on $\mathbb{R}$ ”. Take any n data points $(x_1, y_1), \dots, (x_n, y_n)$.

Observation. *Infinitely many* functions in $\mathcal{F}$ pass through all n points exactly. For each such function, the residual sum of squares is *exactly* zero — the global minimum.

2.2 The Problem — Too Many Solutions 2



f_1, f_2, f_3 all hit every data point exactly: each gives $\text{RSS} = 0$. The optimiser cannot choose among them — they are tied. The problem is **not well-defined**.

2.3 The Lesson

Key takeaway

Minimising the OLS loss over “all functions” is meaningless: infinitely many functions tie at $\text{RSS} = 0$.

To get a unique, sensible estimator, we must **restrict** $\mathcal{F}$ to a smaller subset of well-behaved functions.

The art of non-parametric regression is choosing the restriction. Different restrictions give different methods:

- polynomial regression (§3–§6) — the simplest;
- general basis expansion (§7) — a flexible upgrade;
- smoothing splines, local regression, kernel methods (§8) — more advanced, skipped in this course.

3. Polynomial Spaces

3.1 Polynomials of Degree at Most d

Definition — $\mathcal{P}_d$

The space of **polynomials of degree at most d** is

$$\mathcal{P}_d = \{ f : \mathbb{R} \rightarrow \mathbb{R}, f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d, \beta_j \in \mathbb{R} \}.$$

Notation reminder. Each $f \in \mathcal{P}_d$ is a function $f : \mathbb{R} \rightarrow \mathbb{R}$ specified by its $d + 1$ coefficients $(\beta_0, \beta_1, \dots, \beta_d)$. So $\mathcal{P}_d$ has dimension $d + 1$ — it is a $(d + 1)$ -dimensional subspace of the “all functions” space.

Two extremes:

- $d = 0$: constants $f(x) = \beta_0$.
- $d = 1$: lines $f(x) = \beta_0 + \beta_1 x$ (OLS!).

Larger d gives more flexibility.

3.2 Minimising Over $\mathcal{P}_d$ — Well Defined

Replace “all functions” with “polynomials of degree $\leq d$ ”:

$$\hat{f} = \arg \min_{f \in \mathcal{P}_d} \sum_{i=1}^n (y_i - f(x_i))^2.$$

This problem is well-defined. Why?

- Each $f \in \mathcal{P}_d$ is determined by $\beta \in \mathbb{R}^{d+1}$.
- The objective above is exactly the OLS criterion on a *transformed design matrix* (built next, in §5).
- For $n > d + 1$ and generic x_i , $\mathbf{X}^\top \mathbf{X}$ is invertible. Unique solution exists.

No more ties at zero, no more “infinitely many fits”. The restriction is just the right amount of structure.

3.3 d Is a Hyperparameter

d is a **hyperparameter**

Just like λ in L10, the degree d is a **hyperparameter**: not estimated from the data the way β is. It sits *above* the optimisation and controls the size of the model class.

For now, treat d as **given**. The OLS-on-polynomials solution is well-defined for any choice. At the end (and in Lecture 13) we will come back and show how to pick d from data, again using cross-validation — the same recipe that picks λ for ridge and LASSO.

Two extremes preview:

- d too small $\Rightarrow$ underfit (high bias).
- d too large $\Rightarrow$ overfit (high variance).
- Bias–variance tradeoff again (L10 §3) — same story, different tuning parameter.

4. Why Polynomials? — Two Justifications

4.1 Smooth Functions

Polynomials make sense when the truth f is **smooth**. What do we mean by smooth?

Intuitive definition

A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is **smooth** if it has no sharp corners, no jumps, no breaks — it can be differentiated as many times as we like, and each derivative is itself well-behaved.

Notation reminder. $C^k(\mathbb{R})$ denotes functions with k continuous derivatives; “smooth” usually means C^∞ (infinitely differentiable). The standard examples — $\sin$, $\cos$, $\exp$, polynomials, $\log$ on $(0, \infty)$ — are all smooth.

Two complementary theorems justify polynomial approximation of smooth f :

- **Locally:** Taylor’s theorem (§4.2).
- **Globally:** Stone–Weierstrass theorem (§4.4).

4.2 Local View — Taylor's Theorem

Key Result — Taylor's theorem

If f has $d + 1$ continuous derivatives near $x = a$, then for x near a

$$\begin{aligned} f(x) = & f(a) + f'(a)(x - a) + \frac{f''(a)}{2!}(x - a)^2 \\ & + \cdots + \frac{f^{(d)}(a)}{d!}(x - a)^d + R_d(x). \end{aligned}$$

The first $d + 1$ terms form a *polynomial of degree d in x* ; the remainder $R_d(x)$ goes to zero faster than $(x - a)^d$ as $x \rightarrow a$.

Reading. Near any point a , a smooth f is well approximated by a polynomial of degree d . The approximation improves as d grows, and as x stays close to a .

The truncated polynomial *is* the kind of object we are estimating in $\mathcal{P}_d$.

4.3 Taylor Examples — $\sin$, $\cos$, e^x

Three classic Taylor expansions around $a = 0$:

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

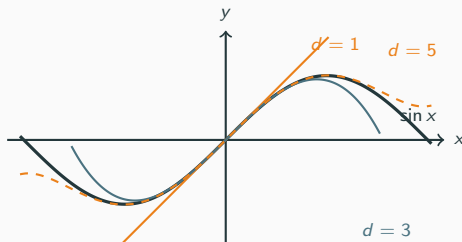
$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

Truncate at degree d : the partial sum lives in $\mathcal{P}_d$.

Picture — approximating $\sin x$.

4.3 Taylor Examples — $\sin$, $\cos$, e^x 2



$d = 1$ tracks $\sin x$ only near 0; $d = 3$ extends out to $\pm\pi/2$; $d = 5$ is essentially indistinguishable from $\sin x$ on $[-\pi, \pi]$.

Moral. Truncate to a finite degree — error is small for x in a bounded interval. The same fits-by-polynomial logic powers $\mathcal{P}_d$ regression.

4.4 Global View — Stone–Weierstrass

Taylor is a *local* statement (good near a fixed point a). A *global* statement is also available.

Key Result — Stone–Weierstrass theorem (informal)

Let $f : [a, b] \rightarrow \mathbb{R}$ be continuous on a closed, bounded interval. For any $\epsilon > 0$, there exists a polynomial $q \in \mathcal{P}_d$ (for some finite d) such that

$$|f(x) - q(x)| < \epsilon \quad \text{for every } x \in [a, b].$$

Polynomials are *dense* in continuous functions on $[a, b]$.

Reading. However bumpy or weird (within “continuous”) the true f is, we can match it *uniformly* on $[a, b]$ to any tolerance ϵ , just by taking d large enough.

4.4 Global View — Stone–Weierstrass 2

Note the trade-off: a smaller ϵ may require a much larger d . The price of accuracy is more parameters — a preview of the bias–variance tradeoff.

4.5 Upshot

Why polynomials are a sensible choice

- **Local** (Taylor): any smooth f looks like a polynomial near every point.
- **Global** (Stone–Weierstrass): any continuous f on $[a, b]$ can be approximated, uniformly, by some polynomial of finite degree.
- **Therefore**: restricting our search from “all functions” (§2, ill-defined) to “polynomials of degree $\leq d$ ” (§3, well-defined) costs us very little in principle — as long as d is large enough to capture the truth and the input is bounded.

In practice, d does not need to be very large. Even $d = 3$ or $d = 4$ captures most smooth shapes that arise in applied work.

5. Polynomial Regression in Practice

5.1 The Polynomial Regression Model

Fix a degree d (treated as given, per §3.3). For $i = 1, 2, \dots, n$, the polynomial regression model is the scalar equation

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \epsilon_i,$$

The observed pairs (X_i, Y_i) are independent across i . Within each observation, $\mathbb{E}[\epsilon_i \mid X_i] = 0$; for the basic standard-error convention, $\text{Var}(\epsilon_i \mid X_i) = \sigma^2$. *These are the same assumptions as the L9 model.*

Notation reminder. The unknowns are the $d + 1$ coefficients $\beta_0, \beta_1, \dots, \beta_d$; the noises ϵ_i are unobserved.

Stack the n scalar equations into vector / matrix form (mirroring L9 §2.1):

$$\underbrace{\mathbf{y}}_{n \times 1} = \underbrace{\mathbf{X}_{\text{poly}}}_{n \times (d+1)} \underbrace{\boldsymbol{\beta}}_{(d+1) \times 1} + \underbrace{\boldsymbol{\epsilon}}_{n \times 1}.$$

5.2 The Four Ingredients in Detail

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \mathbf{X}_{\text{poly}} = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^d \end{pmatrix},$$

$$\boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_d \end{pmatrix}, \quad \boldsymbol{\epsilon} = \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}.$$

- **Response** $\mathbf{y} \in \mathbb{R}^{n \times 1}$: the n observed outcomes.

5.2 The Four Ingredients in Detail 2

- **Design matrix** $\mathbf{X}_{\text{poly}} \in \mathbb{R}^{n \times (d+1)}$: one intercept column plus d transformed predictor columns. Column j contains the $(j - 1)$ -th power x_i^{j-1} . Each *row* lists all powers of one observation x_i ; each *column* is a transformation of the predictor.
- **Coefficients** $\beta \in \mathbb{R}^{(d+1) \times 1}$: the $d + 1$ unknown numbers we want to estimate.
- **Noise** $\epsilon \in \mathbb{R}^{n \times 1}$: conditional mean zero and constant conditional variance σ^2 ; observations are independent across i .

Shape check. $\mathbf{X}_{\text{poly}} \beta$ is $(n \times (d + 1)) \cdot ((d + 1) \times 1) = n \times 1$, matching $\mathbf{y}$. ✓

Row check. Row i of $\mathbf{X}_{\text{poly}} \beta$ is $\beta_0 + \beta_1 x_i + \cdots + \beta_d x_i^d$; adding ϵ_i recovers the scalar equation for y_i . ✓

5.2 The Four Ingredients in Detail 3

Identical shape to the L9 multiple regression equation $\mathbf{y} = \mathbf{X}\beta + \epsilon$ — only the columns of $\mathbf{X}$ have changed.

5.3 OLS Solves It — Almost Nothing New

Apply the L9 formula directly:

Key Result — polynomial regression by OLS

$$\hat{\beta}_{\text{poly}} = (\mathbf{X}_{\text{poly}}^{\top} \mathbf{X}_{\text{poly}})^{-1} \mathbf{X}_{\text{poly}}^{\top} \mathbf{y}.$$

Then the fitted polynomial is

$$\hat{f}(x) = \hat{\beta}_0 + \hat{\beta}_1 x + \hat{\beta}_2 x^2 + \cdots + \hat{\beta}_d x^d.$$

Nothing new at the estimation step. The only change from L9: the columns of $\mathbf{X}_{\text{poly}}$ are powers of one predictor, not different predictors. Standard errors, CIs, the F-test — everything L9 built carries over unchanged.

Software fits this in one line. “Computer does it.”

5.4 Effects of Degree d

Three regimes, three pictures (data drawn from a smooth true curve f^* , fit polynomial of degree d):



- $d = 1$: model too rigid — the line cannot bend. High bias.
- $d = 4$: smooth and tracks the data well.

5.4 Effects of Degree d 2

- $d = 15$: model so flexible it wiggles between data points to chase noise. High variance.

Same bias–variance story as L10 §3 — different tuning parameter, same shape.

5.5 Worked Example — Fabricated Smooth Curve

Setup. Simulated data: $n = 100$ pairs (x_i, y_i) with x_i uniform on $[-3, 3]$ and $y_i = \sin(x_i) + 0.3\epsilon_i$, $\epsilon_i \sim N(0, 1)$. True $f^*(x) = \sin x$.

Software output (fabricated). Fit polynomial regression for $d = 1, 2, 4, 8, 15$ and split into 80 training / 20 validation:

degree d	train RSS	validation MSE	comment
1	48.2	0.51	underfit, line cannot bend
2	46.8	0.48	still too rigid
4	9.7	0.10	sweet spot
8	9.1	0.13	starting to overfit
15	7.4	0.27	overfit, wild between points

5.5 Worked Example — Fabricated Smooth Curve 2

Reading. Train RSS drops monotonically in d , but validation MSE has a clear U-shape with a minimum at $d = 4$. The same picture as L10 §3 in disguise.

How d is picked in practice. Run cross-validation over $d = 1, 2, \dots, d_{\max}$, pick the d with smallest CV error. Same recipe used for λ in L10 (forward pointer to Lecture 13).

6. Multivariate and the Curse of Dimensionality

6.1 Two Original Inputs

Now suppose $\mathbf{x} = (x_1, x_2)$ — $r = 2$ original input variables. The space of polynomials of total degree $\leq d$ in two variables is

$$\mathcal{P}_d^{(2)} = \left\{ \sum_{a+b \leq d} \beta_{ab} x_1^a x_2^b : \beta_{ab} \in \mathbb{R} \right\}.$$

Example: $d = 2$. The basis is

$$\{1, x_1, x_2, x_1^2, x_1x_2, x_2^2\}.$$

Six monomials; $\beta \in \mathbb{R}^6$.

The two original inputs generate $p = 5$ transformed predictor columns: $x_1, x_2, x_1^2, x_1x_2, x_2^2$. With one intercept column, the design matrix therefore has $p + 1 = 6$ columns. OLS still fits it. Same machinery as L9, plus the L9 *interaction* term x_1x_2 (recall L9 §10) now appearing as a natural part of $\mathcal{P}_2^{(2)}$.

6.2 General r Original Predictors — The Number of Terms Explodes

With r original predictors and degree $\leq d$, the basis collects every monomial $x_1^{a_1} x_2^{a_2} \cdots x_r^{a_r}$ with $a_1 + a_2 + \cdots + a_r \leq d$. Each choice of exponents is one basis function.

The key point. We do not need the exact formula. The number of monomials grows **very fast** as r increases.

Intuition. For $d = 2$ already, adding one more predictor brings in:

- the new variable itself,
- its square,
- one interaction term with *each* existing predictor.

The total count therefore rises rapidly as original predictors are added. The next slide makes this concrete.

6.3 The Blow-Up Table

Number of monomials of degree $\leq d$ in r original predictors:

$r \backslash d$	$d = 2$	$d = 3$	$d = 4$	$d = 5$
1	3	4	5	6
5	21	56	126	252
10	66	286	1,001	3,003
20	231	1,771	10,626	53,130
50	1,326	23,426	316,251	$\sim 3,478,761$

Even modest r and d produce parameter counts that dwarf typical sample sizes. Pretending $n = 200$ rows can pin down 1,000 monomials is hopeless.

6.3 The Blow-Up Table 2

Same blow-up shape as best-subset's 2^p from L10 §2.3, same conclusion: a brute-force, fully-flexible non-parametric model is computationally and statistically infeasible in moderate-to-high dimensions.

6.4 The Curse of Dimensionality

Definition — curse of dimensionality

A general phenomenon: in high-dimensional input spaces, function-fitting problems require **exponentially many** parameters — or **exponentially many** data points — to achieve any given approximation accuracy.

Two important caveats.

- This is *not* a flaw of polynomial regression specifically. *Every* non-parametric method (splines, kernels, local regression, ...) faces an analogous explosion.
- It is a *deep mathematical fact* about the geometry of high-dimensional spaces, not a software problem. No clever algorithm dissolves it.

6.4 The Curse of Dimensionality 2

Consequence. Non-parametric methods work beautifully for r small (1, 2, maybe 5). They struggle for r large — exactly where the most interesting modern problems sit (genomics, imaging).

6.5 Mitigation — LASSO and Ridge to the Rescue

Direct way to fight the blow-up: apply L10's tools to the polynomial design matrix.

Build $\mathbf{X}_{\text{poly}}$ with all the monomials. If there are M non-constant monomials, then the design-matrix convention uses $p = M$ predictor columns plus one intercept column. Solve

$$\hat{\boldsymbol{\beta}}_{\text{LASSO}} = \arg \min_{\boldsymbol{\beta}} \left\{ \|\mathbf{y} - \mathbf{X}_{\text{poly}}\boldsymbol{\beta}\|_2^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}.$$

Ridge instead uses $\lambda \sum_{j=1}^p \beta_j^2$. In both cases the intercept β_0 is not penalised.

What this buys us.

6.5 Mitigation — LASSO and Ridge to the Rescue 2

- LASSO zeros out many non-constant monomials — automatic feature selection on the polynomial basis.
- Ridge shrinks all non-constant terms toward zero — stable in the presence of many correlated monomials.
- Both apply even when the number of monomials exceeds n (recall L10 §4.5 — ridge always works).

This is exactly why L10 and L11 sit next to each other: the non-parametric design matrix is just another design matrix, and the shrinkage machinery applies verbatim.

7. General Basis Function Approach

7.1 Polynomials Are Not the Only Choice

Section 4 justified polynomials with two theorems — Taylor (local) and Stone–Weierstrass (global). But *polynomials are not the only family that approximates smooth functions*. Many other function families do the same job, sometimes better.

Examples.

- **Trigonometric series** (sines and cosines) — great for periodic data (seasonal sales, EEG signals, sound waves). Polynomial regression tries to fit a wave with bent rulers — a struggle. Sines were *built* for waves.
- **Piecewise constants / step functions** — good when f has plateaus or jumps (tax brackets, age groups). Polynomials cannot reproduce a flat region cheaply.

7.1 Polynomials Are Not the Only Choice 2

- **Splines (piecewise polynomials)** — a compromise: polynomial inside each interval, smoothly joined at chosen points. The de-facto standard for smooth-but-flexible fits.
- **Indicator functions of features** — $\mathbf{1}\{x > c\}$ for thresholds c . Building block for decision trees (L14).

Different shapes of f call for different building blocks. The **general basis function approach** treats the choice of building blocks as a modelling decision separate from the estimation procedure.

7.2 The General Recipe

To keep the design-matrix convention consistent, r counts the original inputs and p counts the predictor columns actually fitted after transformation. Let x be scalar (or belong to $\mathbb{R}^r$), and pick p transformed predictor functions $\phi_1, \phi_2, \dots, \phi_p$, called the **basis functions**. Fit

$$\hat{f}(x) = \hat{\beta}_0 + \hat{\beta}_1\phi_1(x) + \dots + \hat{\beta}_p\phi_p(x).$$

The design matrix has one intercept column plus the p transformed predictor columns:

$$\mathbf{X} = \begin{pmatrix} 1 & \phi_1(x_1) & \cdots & \phi_p(x_1) \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \phi_1(x_n) & \cdots & \phi_p(x_n) \end{pmatrix} \in \mathbb{R}^{n \times (p+1)}.$$

7.2 The General Recipe 2

Then OLS / ridge / LASSO apply unchanged:

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (\text{or with a penalty, exactly as in L10}).$$

All the L9 / L10 / L11 machinery is silent about what ϕ_j is. It only sees a numeric design matrix. Polynomial regression of degree d is the special case $p = d$ and $\phi_j(x) = x^j$.

7.3 Trigonometric Series — Fourier Intuition

Periodic / cyclical data (seasonal sales, daily temperatures, sound, business cycles) are the natural home of **trigonometric series**, also called **Fourier series**.

The basis. Replace monomials by sines and cosines at increasing frequencies $k = 1, 2, 3, \dots$:

$$1, \sin x, \cos x, \sin(2x), \cos(2x), \sin(3x), \cos(3x), \dots$$

Truncate at frequency K : $\hat{f}(x)$ is a linear combination of 1 plus the first K sine–cosine pairs — $2K + 1$ basis functions in total. (K plays the role of d in polynomial regression: a hyperparameter controlling flexibility.)

In our design-matrix convention, these are $p = 2K$ transformed predictor columns plus one intercept column, for $p + 1 = 2K + 1$ columns in total.

7.3 Trigonometric Series — Fourier Intuition 2

Each pair $(\sin(kx), \cos(kx))$ captures one rate of *oscillation*; a fitted coefficient says “how strong is this oscillation in the data”.

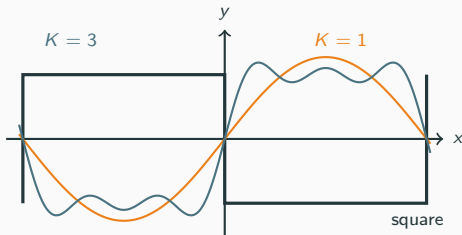
7.4 Why Sines and Cosines Work for Waves

The Fourier idea, in one sentence: any reasonable periodic function is a sum of sine and cosine waves at different frequencies.

Just as Stone–Weierstrass said “smooth functions on $[a, b]$ are sums of polynomials”, a parallel theorem (the **Fourier theorem**) says “periodic functions on $[-\pi, \pi]$ are sums of sines and cosines.”

Concrete picture. A square wave (jumps between -1 and $+1$) can be assembled by adding the first few odd-frequency sine waves:

7.4 Why Sines and Cosines Work for Waves 2



$K = 1$: one wave catches the overall up-and-down pattern. $K = 3$: three waves combine into a much closer approximation to the square shape. Send $K \rightarrow \infty$: the partial sum converges to the square wave (where it is continuous).

7.5 Fourier Regression in Practice

Suppose y is a daily series with weekly periodicity. Rescale time so one week is 2π . Build the Fourier design matrix (one column for the intercept, then a sine/cosine pair for each frequency $k = 1, 2, \dots, K$):

$$\mathbf{X}_{\text{Fourier}} = \begin{pmatrix} 1 & \sin x_1 & \cos x_1 & \cdots & \sin(Kx_1) & \cos(Kx_1) \\ 1 & \sin x_2 & \cos x_2 & \cdots & \sin(Kx_2) & \cos(Kx_2) \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & \sin x_n & \cos x_n & \cdots & \sin(Kx_n) & \cos(Kx_n) \end{pmatrix}.$$

Its shape is $n \times (2K + 1) = n \times (p + 1)$ because the $2K$ sine–cosine columns are the transformed predictors. Run OLS. The fitted $\hat{f}$ is the trigonometric series that best matches the data.

Reading the coefficients.

- $\hat{\beta}_0$ — the average level.

7.5 Fourier Regression in Practice 2

- $(\hat{\beta}_{\sin,k}, \hat{\beta}_{\cos,k})$ — how loud the frequency- k oscillation is.

Same bias–variance story. K is a hyperparameter just like d for polynomials. Too small: cannot capture the wiggles (high bias). Too large: chases noise (high variance). Pick by cross-validation, same recipe.

Why prefer Fourier over polynomials here?

- Polynomials are not periodic — $\hat{f}(x)$ blows up outside the data range, useless for cyclical extrapolation.
- Sines are bounded and periodic — coefficients are stable and have a clean reading as frequency content.

7.6 Take Away — One Recipe, Many Bases

The basis-function paradigm

Choose predictor features $\phi_1, \dots, \phi_p$ that match the shape of your problem:

- smooth and bounded $\Rightarrow$ polynomials,
- periodic / cyclical $\Rightarrow$ trigonometric (Fourier),
- piecewise constant / threshold $\Rightarrow$ indicators,
- smooth but flexible $\Rightarrow$ splines.

Once the basis is fixed, you are back in familiar OLS / ridge / LASSO territory. Different basis, same machinery.

8. What We Skipped

8.1 Other Non-Parametric Techniques

ISLR Chapter 7 covers a much broader toolkit than we have built. Most use the same conceptual recipe (restrict the function space, tune a hyperparameter, manage bias–variance) but with different function-space restrictions:

- **Regression splines.** Piecewise polynomials joined smoothly at *knots*; the knot locations and the polynomial degree are the hyperparameters.
- **Smoothing splines.** Minimise RSS plus an integrated second-derivative penalty $\int [f''(x)]^2 dx$ — a different kind of shrinkage, penalising *wiggleness* instead of coefficient size.
- **Local regression / LOESS.** For each test point x_0 , fit a polynomial to nearby training points only.

8.1 Other Non-Parametric Techniques 2

- **Kernel methods.** Average response values, weighted by closeness to x_0 .
- **Generalised additive models (GAMs).** Sum of univariate non-parametric fits, one per predictor.

8.2 Why We Skip Them

These methods are conceptually parallel to polynomial regression — same bias–variance picture, same hyperparameter framing — but their derivations and computational details require:

- more linear algebra (smoothing-spline matrix algebra),
- more analysis (kernel theory, Mercer’s theorem),
- heavier software tooling (LOESS bandwidths, GAM back-fitting).

All beyond the math floor of this course.

Where to learn more. ISLR Chapter 7 covers regression splines, smoothing splines, local regression, and GAMs at a level consistent with the rest of the textbook. The conceptual scaffolding we built today applies to all of them.

9. OLS vs Shrinkage vs Polynomial Regression

9.1 Recall the Bias–Variance Framework

From L10 §3, for any target θ and estimator $\hat{\theta}$,

$$\mathbb{E}[(\hat{\theta} - \theta)^2] = \underbrace{(\mathbb{E}[\hat{\theta}] - \theta)^2}_{\text{Bias}^2} + \underbrace{\text{Var}(\hat{\theta})}_{\text{Variance}}.$$

No re-derivation here — the lecture that owns this identity is L10. We only *apply* it.

Three methods to summarise:

- **OLS** (§9.2) — linear regression.
- **Shrinkage** (§9.3) — ridge and LASSO.
- **Polynomial regression** (§9.4) — this lecture.

One framework, three methods, each with its own tuning parameter (or none, for OLS). Side-by-side table in §9.5.

9.2 OLS in the Bias–Variance Framework

- **Bias.** Zero when the linear model is correct (L7 §3 establishes unbiasedness for $\hat{\beta}_1$; same logic for multiple regression):
 $\mathbb{E}[\hat{\beta}_{\text{OLS}}] = \beta$.
- **Variance.** $\text{Var}(\hat{\beta}_{\text{OLS}}) = \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$. Large when p is large relative to n , or when $\mathbf{X}^\top \mathbf{X}$ is near-singular (recall L10 §1.4 Case B).
- **Tuning parameter.** None. OLS has no hyperparameter — bias and variance are whatever the data and the linear model deliver.

Where OLS wins. The classical regime: n much larger than $p + 1$ and the truth is approximately linear. Low bias, manageable variance, no tuning needed.

Where OLS struggles. Either when the truth is curved (high bias — a straight line cannot bend, motivating polynomials), or when p is large relative to n (high variance — motivating shrinkage).

9.3 Shrinkage (Ridge / LASSO) in the Bias–Variance Framework

- **Bias.** *Positive* for any $\lambda > 0$. Grows with λ : bigger penalty $\Rightarrow$ more shrinkage toward zero $\Rightarrow$ more bias.
- **Variance.** Shrunk by λ (L10 §4.7). LASSO additionally drives some slope / basis coefficients exactly to zero, reducing variance even further by dropping noisy predictors.
- **Tuning parameter.** $\lambda \geq 0$. Hyperparameter, picked by cross-validation (L13). $\lambda = 0$ recovers OLS; large λ shrinks every non-intercept coefficient toward zero while leaving β_0 unpenalised.

9.3 Shrinkage (Ridge / LASSO) in the Bias–Variance Framework 2

Where shrinkage wins. When OLS has high variance — p large relative to n , or many noisy or correlated predictors. Ridge stabilises the slope / basis coefficients; LASSO additionally selects variables.

Where shrinkage struggles. If the true function is fundamentally non-linear, shrinkage on a linear design matrix only shrinks the wrong coefficients — the bias is intrinsic to the linear model, not to the size of β .

9.4 Polynomial Regression in the Bias–Variance Framework

- **Bias.** *Decreases* as the degree d grows. Small d is too rigid to bend with the data (Taylor: the truncation error in R_d is large); large d can match almost any smooth f (Stone–Weierstrass).
- **Variance.** *Grows* with d . A degree- d polynomial has $d + 1$ coefficients to estimate from the same n data points — more coefficients means each is estimated less precisely. In high dimensions the curse hits — the monomial count explodes with the number r of original predictors (§6.3).
- **Tuning parameter.** d . Hyperparameter, picked by cross-validation (L13) — same procedure as λ .

9.4 Polynomial Regression in the Bias–Variance Framework 2

Where polynomial regression wins. When the truth is non-linear but r is small. A degree-3 or degree-4 polynomial in one predictor handles most smooth shapes.

Where polynomial regression struggles. High dimensions: the curse (§6.4) makes the design matrix explode. Mitigation: layer LASSO / ridge on top (§6.5) — combines the strengths of this section and §9.3.

9.5 Side-by-Side Comparison

Method	Bias source	Variance source	Tuning param.	Tuned by
OLS	misspecification	$\sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$	—	—
Shrinkage	$\lambda > 0$ shrinkage	shrunk by λ	λ	CV (L13)
Polynomial	d too small	d too large	d	CV (L13)

One framework, three methods, three tuning parameters:

- OLS has no tuning parameter.
- Shrinkage tunes λ .
- Polynomial regression tunes d .

Pick the best value by cross-validation (L13).

10. Summary

10.1 Take Away — Foundations

Three big ideas

- **Parametric vs non-parametric.** Parametric models index f by a finite vector θ ; non-parametric models let f live in an infinite-dimensional space.
- **Restricting the function space.** Minimising over all functions is ill-defined. Polynomials of degree $\leq d$ give a finite-dimensional space where the OLS recipe works unchanged.
- **Polynomials approximate smooth functions.** Locally by Taylor, globally by Stone–Weierstrass — so the restriction loses little.
- **Curse of dimensionality.** The number of monomials explodes with the number r of original predictors — a deep mathematical fact, not a methods problem.
- **Shrinkage to the rescue.** LASSO / ridge applied to the polynomial design matrix mitigate the blow-up. L10 and L11 combine cleanly.

10.2 Take Away — Key Formulas

The L11 toolbox

- **Polynomial space:** $\mathcal{P}_d = \{ \beta_0 + \beta_1 x + \cdots + \beta_d x^d \}$.
- **Polynomial design matrix:** $\mathbf{X}_{\text{poly}} \in \mathbb{R}^{n \times (d+1)}$ with row i entries $(1, x_i, x_i^2, \dots, x_i^d)$.
- **Polynomial OLS:** $\hat{\beta}_{\text{poly}} = (\mathbf{X}_{\text{poly}}^\top \mathbf{X}_{\text{poly}})^{-1} \mathbf{X}_{\text{poly}}^\top \mathbf{y}$.
- **Monomial count** for degree $\leq d$ in r original predictors: grows very fast — exponentially in r — and drives the curse of dimensionality (§6.3 table).
- **Bias–variance identity** (from L10 §3):
$$\mathbb{E}[(\hat{\theta} - \theta)^2] = (\mathbb{E}[\hat{\theta}] - \theta)^2 + \text{Var}(\hat{\theta}).$$
- **Hyperparameter d :** treat as given, pick by cross-validation (L13).

10.3 Where Each Idea Reappears

Forward pointers

- **Lecture 12 (logistic regression)**. Same loss-minimisation template applied to binary $Y \in \{0, 1\}$. Polynomial / basis expansion carries over verbatim: “logistic regression with polynomial features” is exactly the construction of §5 plugged into a different loss.
- **Lecture 13 (resampling)**. Cross-validation picks the hyperparameter — λ for shrinkage, d for polynomial regression, basis size for general bases. One procedure, many uses.
- **Lecture 14 (random forests)**. A fundamentally different non-parametric idea: instead of fixing a basis, the method partitions the input space *recursively* and averages locally. Side-steps the curse of dimensionality by adapting the partition to the data.